CSC343H5 Introduction to Databases

Sprint 4: Normalization

Student Submission Template

Full name: Abdul Hadi Khan
Student number: 1007984736
UTORid: khanab72
Course: CSC343H5 Introduction to Databases
Sprint: 4: Normalization
Submission: First submission

How to use this template. Write each answer beneath its prompt, replacing the grey *[placeholder]* text. Use `\fd` for the arrow (`A` → `B`), `\clo{...}` for a closure (`{A}`$^+$), and `\pk{...}` to underline a primary-key attribute. Show every step; closures and decomposition steps are where the marks are.

Reference: `research_flat`. Key (`grant_code`, `project_id`, `researcher_id`). Attributes: `grant_code`, `grant_title`, `funding_body`, `funding_sector`, `project_id`, `project_title`, `budget`, `researcher_id`, `researcher_name`, `institution`, `role_on_project`. Given FDs:

- FD1: `grant_code` → `grant_title`, `funding_body`
- FD2: `project_id` → `project_title`, `budget`
- FD3: `researcher_id` → `researcher_name`, `institution`
- FD4: `funding_body` → `funding_sector`
- FD5: `grant_code`, `project_id`, `researcher_id` → `role_on_project`

1. Task 1: Closure & Candidate Key

(a) Closure of the composite key

Let $K = \{\texttt{grant\_code}, \texttt{project\_id}, \texttt{researcher\_id}\}$. Show each step; the final set must contain every attribute.

$$
\begin{aligned}
K^+ &= \{\texttt{grant\_code}, \texttt{project\_id}, \texttt{researcher\_id}\} \qquad \text{(start)} \\
&= K^+ \cup \{\texttt{grant\_title}, \texttt{funding\_body}\} \\
&\quad \text{(by FD1: } \texttt{grant\_code} \to \texttt{grant\_title}, \texttt{funding\_body}) \\
&= K^+ \cup \{\texttt{project\_title}, \texttt{budget}\} \\
&\quad \text{(by FD2: } \texttt{project\_id} \to \texttt{project\_title}, \texttt{budget}) \\
&= K^+ \cup \{\texttt{researcher\_name}, \texttt{institution}\} \\
&\quad \text{(by FD3: } \texttt{researcher\_id} \to \texttt{researcher\_name}, \texttt{institution}) \\
&= K^+ \cup \{\texttt{funding\_sector}\} \\
&\quad \text{(by FD4: } \texttt{funding\_body} \to \texttt{funding\_sector}\text{, since } \texttt{funding\_body} \text{ is now in the set)} \\
&= K^+ \cup \{\texttt{role\_on\_project}\} \\
&\quad \text{(by FD5: } K \to \texttt{role\_on\_project}) \\
&= \text{all 11 attributes} \qquad \text{(done)}
\end{aligned}
$$

(b) Candidate key

$K = \{$grant_code, project_id, researcher_id$\}$ is the only candidate key. None of the other eight attributes (grant_title, funding_body, funding_sector, project_title, budget, researcher_name, institution, role_on_project) ever appears on the left-hand side of an FD, so no key can be built using them, and dropping any one of grant_code, project_id, researcher_id from K leaves the closure missing the attributes owned by the id that was dropped (e.g. $\{$project_id, researcher_id$\}^+$ never reaches grant_title or funding_body), so all three are required.

(c) Prime and non-prime attributes

Prime: grant_code, project_id, researcher_id **Non-prime:** grant_title, funding_body, funding_sector, project_title, budget, researcher_name, institution, role_on_project

2. Task 2: Minimal Cover

Step 1: Single-attribute right-hand sides

Splitting each FD's right-hand side into single attributes gives 8 FDs:

- grant_code → grant_title grant_code → funding_body
- project_id → project_title project_id → budget
- researcher_id → researcher_name researcher_id → institution
- funding_body → funding_sector
- grant_code, project_id, researcher_id → role_on_project

Step 2: Remove redundant left-hand-side attributes

Only FD5 (grant_code, project_id, researcher_id → role_on_project) has more than one attribute on the left. We test whether each attribute can be dropped by computing the closure of the *other two* using the full FD set from Step 1:

$$\{\texttt{project\_id}, \texttt{researcher\_id}\}^+$$

reaches project_title, budget, researcher_name, and institution, but never role_on_project ⇒ grant_code is **not** redundant.

$$\{\texttt{grant\_code}, \texttt{researcher\_id}\}^+$$

reaches grant_title, funding_body, funding_sector, researcher_name, and institution, but never role_on_project ⇒ project_id is **not** redundant.

$$\{\texttt{grant\_code}, \texttt{project\_id}\}^+$$

reaches grant_title, funding_body, funding_sector, project_title, and budget, but never role_on_project ⇒ researcher_id is **not** redundant. All three attributes are needed on the left of FD5, so nothing is removed in this step.

Step 3: Remove redundant FDs

For each FD $X \to Y$ from Step 1, we compute $\{X\}^+$ using the *other seven* FDs and check whether Y is still reached:

- Drop `grant_code` $\to$ `grant_title`: $\{$`grant_code`$\}^+$ reaches only `funding_body` and `funding_sector` $\Rightarrow$ `grant_title` not reached, **keep**.
- Drop `grant_code` $\to$ `funding_body`: $\{$`grant_code`$\}^+$ reaches only `grant_title` $\Rightarrow$ `funding_body` not reached, **keep**.
- Drop `project_id` $\to$ `project_title`: $\{$`project_id`$\}^+$ reaches only `budget` $\Rightarrow$ `project_title` not reached, **keep**.
- Drop `project_id` $\to$ `budget`: $\{$`project_id`$\}^+$ reaches only `project_title` $\Rightarrow$ `budget` not reached, **keep**.
- Drop `researcher_id` $\to$ `researcher_name`: $\{$`researcher_id`$\}^+$ reaches only `institution` $\Rightarrow$ `researcher_name` not reached, **keep**.
- Drop `researcher_id` $\to$ `institution`: $\{$`researcher_id`$\}^+$ reaches only `researcher_name` $\Rightarrow$ `institution` not reached, **keep**.
- Drop `funding_body` $\to$ `funding_sector`: $\{$`funding_body`$\}^+$ reaches nothing new $\Rightarrow$ `funding_sector` not reached, **keep**.
- Drop FD5: without `grant_code`, `project_id`, `researcher_id` $\to$ `role_on_project`, the closure of that same set still reaches every other attribute through FD1–FD4, but never `role_on_project` $\Rightarrow$ **keep**.

No FD is redundant, so all 8 survive.

Resulting minimal cover

The minimal cover turns out to contain the same information as the original FD set in §3 (grouping same-left-hand-side FDs back together for readability):

$$\text{Minimal cover} = \begin{cases} \texttt{grant\_code} \to \texttt{grant\_title}, \texttt{funding\_body} \\ \texttt{project\_id} \to \texttt{project\_title}, \texttt{budget} \\ \texttt{researcher\_id} \to \texttt{researcher\_name}, \texttt{institution} \\ \texttt{funding\_body} \to \texttt{funding\_sector} \\ \texttt{grant\_code}, \texttt{project\_id}, \texttt{researcher\_id} \to \texttt{role\_on\_project} \end{cases}$$

3. Task 3: BCNF Decomposition

For each step, name the violating FD, give the two resulting relations with their keys, and show the lossless check. Add as many steps as you need.

Decomposition steps

Step 1. Decompose `research_flat` on `grant_code` $\to$ `grant_title`, `funding_body`.

- **Violation:** `grant_code` is not a superkey of `research_flat`. $\{$`grant_code`$\}^+$ reaches only `grant_title`, `funding_body`, and (via FD4) `funding_sector` - it never reaches `project_id` or researcher_id, so it does not reach all 11 attributes.

- **Resulting relations:** R1(grant_code, grant_title, funding_body, funding_sector) with key grant_code (funding_sector is included because grant_code's closure reaches it transitively through funding_body); R2(grant_code, project_id, project_title, budget, researcher_id, researcher_name, institution, role_on_project) with key grant_code, project_id, researcher_id.
- **Lossless check:** shared attributes = {grant_code}; grant_code is a key of R1, so the split is lossless.

Step 2. Decompose R1 on funding_body → funding_sector.

- **Violation:** funding_body is not a superkey of R1. $\{$funding_body$\}^+$ reaches only funding_sector - it never reaches grant_code, the key of R1.
- **Resulting relations:** funder(funding_body, funding_sector) with key funding_body; grant(grant_code, grant_title, funding_body) with key grant_code.
- **Lossless check:** shared attributes = {funding_body}; funding_body is a key of funder, so the split is lossless.

Step 3. Decompose R2 on project_id → project_title, budget.

- **Violation:** project_id is not a superkey of R2. $\{$project_id$\}^+$ reaches only project_title and budget - it never reaches grant_code or researcher_id, part of R2's key.
- **Resulting relations:** project(project_id, project_title, budget) with key project_id; R2b(grant_code, project_id, researcher_id, researcher_name, institution, role_on_project) with key grant_code, project_id, researcher_id.
- **Lossless check:** shared attributes = {project_id}; project_id is a key of project, so the split is lossless.

Step 4. Decompose R2b on researcher_id → researcher_name, institution.

- **Violation:** researcher_id is not a superkey of R2b. $\{$researcher_id$\}^+$ reaches only researcher_name and institution - it never reaches grant_code or project_id, part of R2b's key.
- **Resulting relations:** researcher(researcher_id, researcher_name, institution) with key researcher_id; participation(grant_code, project_id, researcher_id, role_on_project) with key grant_code, project_id, researcher_id.
- **Lossless check:** shared attributes = {researcher_id}; researcher_id is a key of researcher, so the split is lossless.

No further FDs are violated in any of the five relations below, so the decomposition is complete.

Final BCNF schema

- funder(funding_body, funding_sector)
- grant(grant_code, grant_title, funding_body)
- project(project_id, project_title, budget)

- `researcher(researcher_id, researcher_name, institution)`
- `participation(grant_code, project_id, researcher_id, role_on_project)`

(`grant.funding_body` is a foreign key into `funder`; `participation`'s three key columns are foreign keys into `grant`, `project`, and `researcher` respectively.)

BCNF verification

In each relation, the left-hand side of the only non-trivial FD that applies is exactly that relation's primary key, so every non-trivial FD has a superkey left-hand side:

- `funder`: `funding_body` → `funding_sector`, and `funding_body` is the key.
- `grant`: `grant_code` → `grant_title`, `funding_body`, and `grant_code` is the key.
- `project`: `project_id` → `project_title`, `budget`, and `project_id` is the key.
- `researcher`: `researcher_id` → `researcher_name`, `institution`, and `researcher_id` is the key.
- `participation`: `grant_code`, `project_id`, `researcher_id` → `role_on_project`, and this combination is the whole key.

All five relations are in BCNF.

4. Task 4: 3NF Check & Justification

(a) Dependency preservation

Each FD in the minimal cover can be checked entirely inside one of the five final relations, with no join required:

- `grant_code` → `grant_title`, `funding_body` - this is checkable in `grant`.
- `project_id` → `project_title`, `budget` - this is checkable in `project`.
- `researcher_id` → `researcher_name`, `institution` - this is checkable in `researcher`.
- `funding_body` → `funding_sector` - this is checkable in `funder`.
- `grant_code`, `project_id`, `researcher_id` → `role_on_project` - checkable in `participation`.

Since every FD lands cleanly in exactly one relation, this BCNF decomposition **is dependency-preserving**. A 3NF synthesis is not needed here; the BCNF schema above can be used directly, since it gives both no redundancy (BCNF) and dependency preservation, which is not always possible together but happens to hold for this FD set.

(b) Anomaly example

Update anomaly. In `research_flat`, grant `NSERC-4471` appears on two rows: (`NSERC-4471`, `EcoForecast`, Amara Osei, PI) and (`NSERC-4471`, `EcoForecast`, Priya Sharma, Postdoc). The value of `grant_title` is copied onto both rows. If the grant is renamed, both rows must be updated; if only one is changed, the table now claims `NSERC-4471` has two different titles at once, an inconsistency that cannot happen in the decomposed schema, where `grant_title` is stored exactly once, in the `grant` table, keyed by `grant_code`.

5. AI Usage Declaration

I used an AI tool while completing this sprint, as declared below.

- **Tool:** Claude Sonnet 5

Interactions:

1. I gave Claude the sprint handout and asked it to guide me step by step since I was new/confused about this topic.
2. I asked it to re-explain closure and candidate-key computation from scratch in simpler terms, since I didn't follow the reasoning the first time.
3. I gave it several lines and formulas which were running past the page margin in the compiled PDF and asked Claude to fix the LaTeX formatting.
4. I asked Claude to check the compiled PDF, both for correctness of the answers and for formatting, before I did my own final manual review.

How I verified / applied it: After the above, I reviewed the entire compiled PDF manually myself, re-checking the closure, the minimal-cover redundancy tests, and the BCNF decomposition steps against the FDs in §3 to confirm each step and result before including it here.